TD Targets

$$VE = \mathbb{E}_{\pi}\left[\left(v_{\pi}(S_t) - \hat{v}(S_t, \bar{w})\right)^2\right]$$

Use sample approx

$$\left(v_{\pi}(S_t) - \hat{v}(S_t, \bar{w})\right)^2$$

Replace v_π by target

- Bootstrapping based target

$$U(S_t) = R_{t+1} + \hat{v}(S_{t+1}, \mathbf{w}_t)$$

- Target is a function of ***w***

$$\left(R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t)\right)^2$$

is a noisy approximation of $\mathbb{E}_{\pi}\left[\left(v_{\pi}(S_t) - \hat{v}(S_t, w_t)\right)^2\right]$

- We need to calculate derivative wrt the target too

$$2\left(R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)\right)\left(\gamma \nabla \hat{v}(S_{t+1}, w) - \nabla \hat{v}(S_t, w)\right)$$

calculated at $w = w_t$.

Semi-Gradient TD

- Assume target doesn't change for small changes in ***w***
- At time step *t* the semi-gradient is

$$-2(R_{t+1} + \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t))\nabla_{\mathbf{w}_t} v(S_t, \mathbf{w}_t)$$

- The semi-gradient weight update step becomes

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t(R_{t+1} + \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t))\nabla_{\mathbf{w}_t} v(S_t, \mathbf{w}_t)$$

Semi-Gradient SARSA

1: Initialize $\mathbf{w}$
2: **while** true **do**
3: Initialize step $t = 0$ at beginning of a new episode
4: Initial state S_0, action A_0, reward R_1
5: **for** step t in an episode **do**
6:

$$A_{t+1} \sim \pi(a|S_{t+1}) = \begin{cases} 1-\epsilon & a \in \arg\max_j \hat{Q}(S_{t+1}, j, \mathbf{w}_t) \\ \epsilon/|A(S_{t+1})| & a \in A(S_{t+1}) \end{cases}$$

7: $\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t(R_{t+1} + \hat{Q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{Q}(S_t, A_t, \mathbf{w}_t))\nabla_{\mathbf{w}_t}\hat{Q}(S_t, A_t, \mathbf{w}_t)$
8: Agent excutes action A_{t+1}, gets R_{t+2}, and the new state is S_{t+2}
9: $t = t + 1$

Semi-Gradient Q-Learning

1: Initialize $\mathbf{w}$
2: **while** true **do**
3: Initialize step $t = 0$ at beginning of a new episode
4: Initial state S_0, action A_0 using ϵ-greedy, reward R_1
5: **for** step t in an episode **do**

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t(R_{t+1} + \gamma \max_{a \in A(S_{t+1})} \hat{Q}(S_{t+1}, a, \mathbf{w}_t) - \hat{Q}(S_t, A_t, \mathbf{w}_t))\nabla_{\mathbf{w}_t} \hat{Q}(S_t, A_t, \mathbf{w}_t)$$

6: Agent chooses A_{t+1} using ϵ-greedy policy
7: $t = t + 1$

Linear Approximation Methods

- MC (Prediction)
  - Convergence [To global optimum $\min_{\bar{w}} VE(\bar{w})$]
- TD (Prediction)
  - Convergence, TD fixed point (LSTD)
- Examples
  - ↳ Polynomial Features } Ways of setting
  - ↳ Coarse Coding. } matrix Φ $|S| \times d$.

Consider the semi-gradient update step for the prediction problem:

We want to find the limit $\bar{v}$, as in the w that minimizes the value error.

$$w_{t+1} = w_t + \alpha_t \left(R_{t+1} + \gamma \hat{v}(S_{t+1}; w_t) - \hat{v}(S_t; w_t)\right) \nabla_w \hat{v}(S_t; w)\Big|_{w=w_t}$$

What's the optimization problem corresponding to the above gradient descent step?

Minimize $\overline{w}$ $\quad E\left[\left(R_{t+1} + \gamma \hat{v}(S_{t+1}; w_t) - \hat{v}(S_t; \overline{w})\right)^2\right]$

To solve this exactly, find $\overline{w}$ that satisfies $\nabla E\left[\left(R_{t+1} + \gamma \hat{v}(S_{t+1}; w_t) - \hat{v}(S_t; \overline{w})\right)^2\right] = 0$

$$\Rightarrow E\left[2\left(R_{t+1} + \gamma \hat{v}(S_{t+1}; w_t) - \hat{v}(S_t; w)\right) \nabla \hat{v}(S_t; w)\right] = 0$$

w_{t+1} solves this.

Assuming a linear approximation:

$$\hat{v}(S_t; w) = \overline{x}(S_t)^T_{1\times d} w_{d\times 1}$$

$$\nabla \hat{v}(S_t; w) = \overline{x}(S_t)_{d\times 1}$$

$$\underline{\Phi} = \begin{bmatrix} \vdots \\ \overline{x}(S_t) \\ \vdots \end{bmatrix}$$

Recall the gradient bandit algorithm.

We wanted the step updates when using stochastic gradient descent to be the same as the steps updates of the exact problem (the above minimization problem).

The exact problem for when we are using linear approximation $\underline{\Phi} w$ is solved at w_{TD}.

At $w_t = w_{TD}$ we have:

For linear approx:

$$\nabla_{w_t} \hat{v}(S_t; w_t) =$$

The true gradient is

$$E\left[\left(R_{t+1} + \gamma \overline{x}(S_{t+1})^T w_{TD} - \overline{x}(S_t)^T w_{TD}\right) \overline{x}(S_t)\right]$$

Solve $E\left[\left(R_{t+1} + \gamma \overline{x}(S_{t+1})^T w_{TD} - \overline{x}(S_t)^T w_{TD}\right) \overline{x}(S_t)\right] = 0$

to obtain w_{TD}.

Back to the SGD update for linear approximation:

$$w_{t+1} = w_t + \alpha \left(R_{t+1} + \gamma\, x(S_{t+1})^T w_t - x(S_t)^T w_t\right) x(S_t)$$

In "steady state"

$$E[w_{t+1} \mid w_t] = w_t + \alpha\, E\left[\left(R_{t+1} + \gamma\, x(S_{t+1})^T w_t - x(S_t)^T w_t\right) x(S_t)\right]$$

w_{TD} is a fixed-point of the above equation.

Why??

TD semi-gradient converges to w_{TD} in case of linear approx. [Convergence proof in book]

Also, [see book]

$$\overline{VE}(w_{TD}) \leq \frac{1}{1-\gamma} \min_{w} \overline{VE}(\hat{v})$$

Examples of linear Approximation Architectures.

POLYNOMIAL BASIS

Consider any state $\bar{s}$. Suppose it is a $k \times 1$ vector.

$$\bar{s} = \begin{bmatrix} s_1 & s_2 & \cdots & s_k \end{bmatrix}^T$$

We want to map any state $\bar{s}$ to a corresponding $1 \times d$ feature vector $\bar{x}(\bar{s})$, which will be a row in $\Phi_{|S| \times d}$.

Order-n polynomial basis feature vector:

Feature

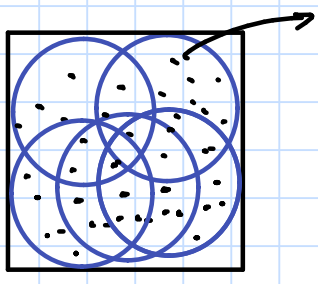
$$x_i(\bar{s}) = \prod_{j=1}^{k} s_j^{c_{ij}}, \quad c_{ij} \in \{0, 1, 2, \ldots, n\}$$

How many possible different features?

$$(n+1)^k$$

COARSE CODING.

2-D state space ($k=2$)



Size of the feature vector is (no. of circles × 1).

Element x_i corresponds to the i^{th} circle.

$x_i = 1$ if the state lies in the i^{th} circle.

The linear approximation is

$\hat{v}(s, \bar{w})$.

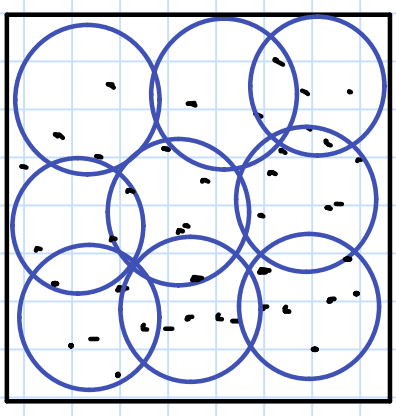
Which elements of the weight vector $\bar{w}$ are updated, for a given state s?

$$\bar{w}_{t+1} = \bar{w}_t + \alpha \; (\quad) \nabla_{\bar{w}} \hat{v}(s, \bar{w})\Big|_{\bar{w}=\bar{w}_t}$$

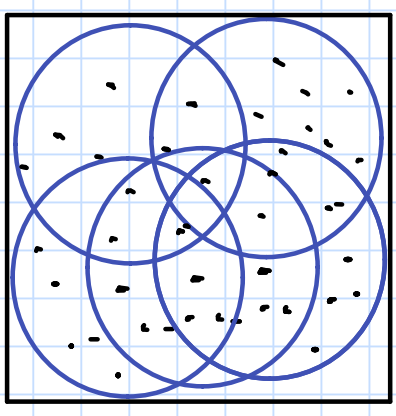
↳ What is this equal to for a linear approximation?

Generalization

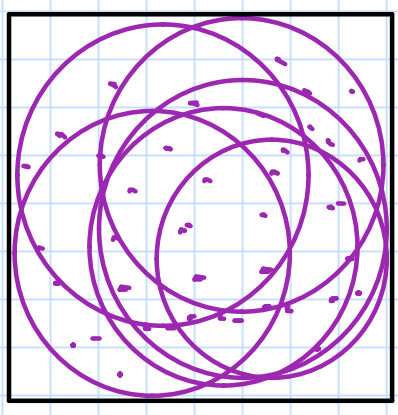
How much does the updation of the weight vector on observing a state impact the values of other states?



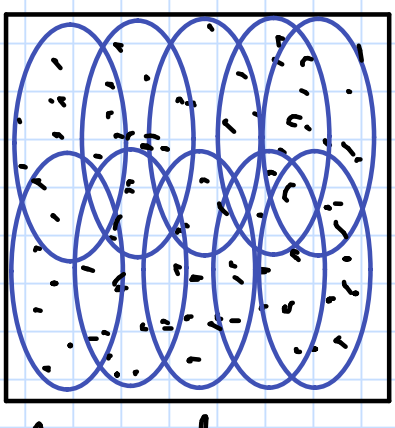
Narrow Generalization



Broad Generalization

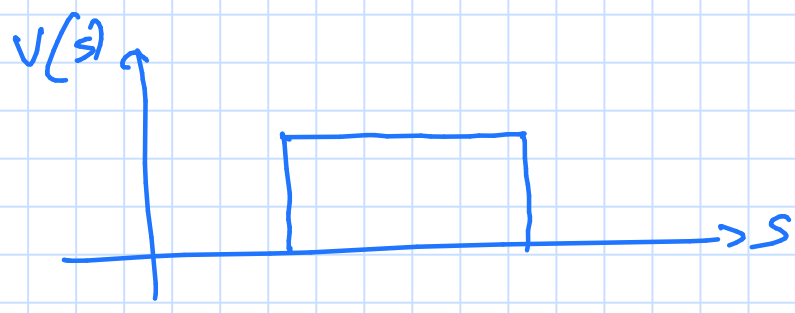


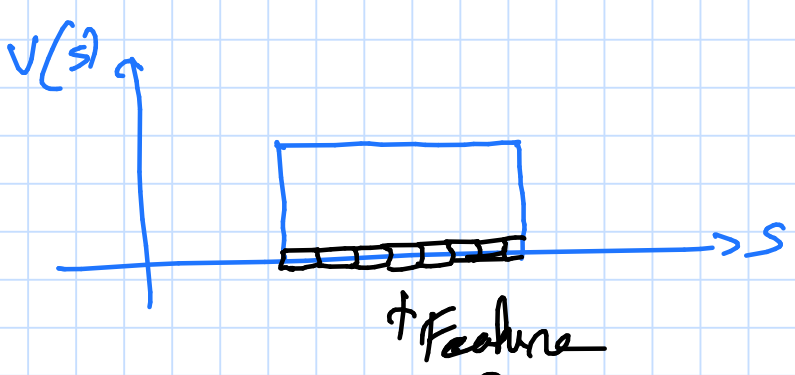
Even more than the previous.

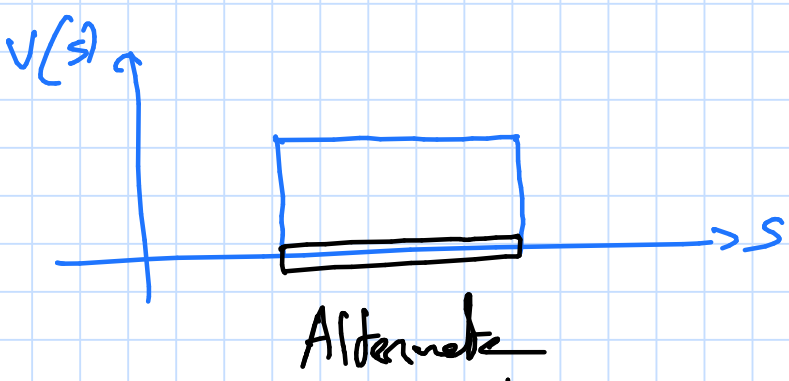


Asymmetric Generalization

Example: (Impact of Gen on value update).







So Far: Linear Approximation Methods

- Polynomial basis
- Coarse Coding
  - Tiling

CHOOSING STEP SIZES

Sufficient to satisfy:

$$\sum \alpha_t = \infty$$

$$\sum \alpha_t^2 < \infty.$$

Sample mean uses $\frac{1}{t}$

↳ Not good for non-stationary problems / TD methods / Fn approx based methods.

Rules of thumb:

Consider update for the tabular case:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma Q(S_{t+1}, \operatorname{argmax}_a Q(S_{t+1}, a)) - Q(S_t, A_t) \right)$$

$\alpha = 1 \Rightarrow$

$\alpha = \frac{1}{t} \Rightarrow$

$$Q^{(1)} = Q^{(0)} + \left(T^{(1)} - Q^{(0)} \right)$$

$$Q^{(2)} = Q^{(1)} + \frac{1}{2}\left(T^{(2)} - Q^{(1)} \right)$$

$$Q^{(3)} = Q^{(2)} + \frac{1}{3}\left(T^{(3)} - Q^{(2)} \right)$$

$$= Q^{(0)}(S_t, A_t) + \frac{1}{2}\left(\text{Target}^{(1)} - Q^{(0)}(S_t, A_t) \right)$$

$$+ \frac{1}{2}\left(\text{Target}^{(2)} - Q^{(0)}(S_t, A_t) - \frac{1}{2}\left(\text{Target}^{(1)} - Q^{(0)}(S_t, A_t) \right) \right)$$

$$= \frac{1}{2}\text{Target}^{(1)} + \frac{1}{2}\text{Target}^{(2)} - \frac{1}{4}\text{Target}^{(1)} + \frac{1}{2} Q^{(0)}(S_t, A_t)$$

Consider the SGD Update: (linear Approx)

$$w_{t+1} = w_t + \alpha_t \underbrace{\left(\text{Target} - Q(S_t, a_t) \right)}\ \underbrace{\overline{x_t}}$$

$$\alpha_t = \frac{1}{t}\left(\frac{1}{E\left[\overline{x_t}^T x_t \right]} \right)$$

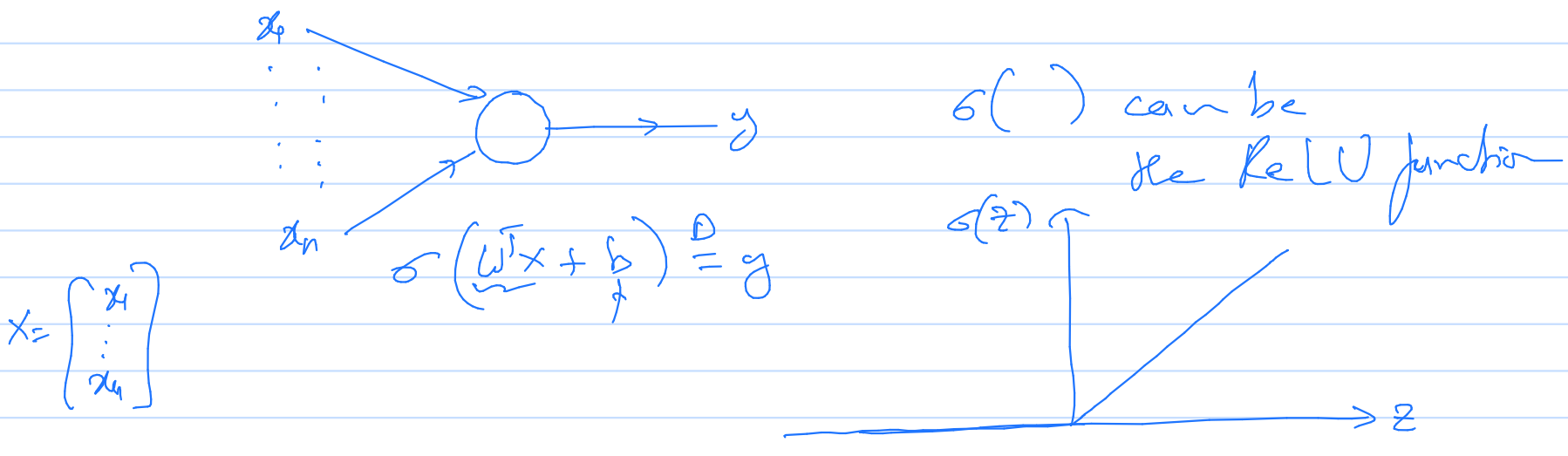
Non-Linear Architectures

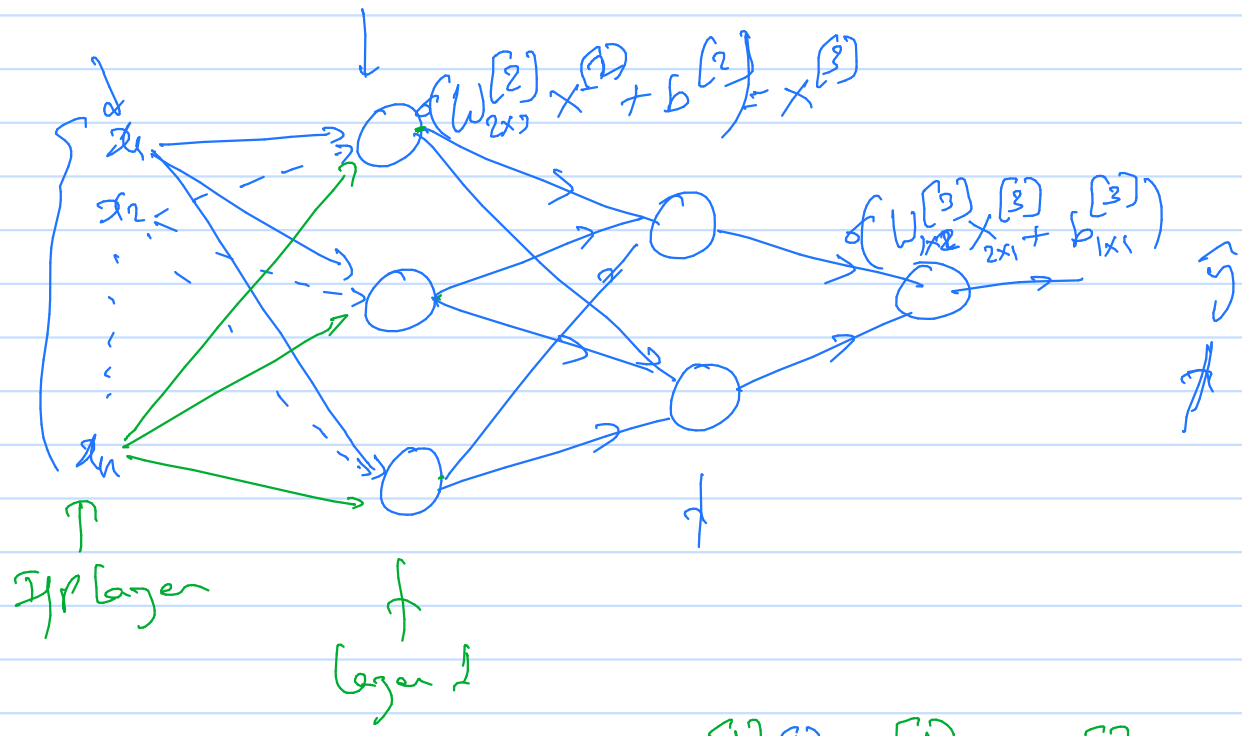
- Multi-layer Perceptrons/ feed forward neural networks/ feed forward artificial neural networks

- Backpropagation

SALIENT FEATURES OF NEURAL NETS.

1. Cost function.
2. Forward Pass
3. Backward Pass
4. Activation function
5. Hyper parameters





O/P of layer 1 is $W^{[1]}_{m\times n} x^{(i)}_{n\times 1} + b^{[i]}_{m\times 1} \triangleq x^{[2]}$

Forward Pass gives the output for a given input.

Loss fn: $L(\hat{y}, y) = -\left(y \log \hat{y} + (1-y)\log(1-\hat{y})\right)$ [for a given sample in the training set]

(image, category)

In total m samples in the training set.

Cost fn: $\frac{1}{m} \sum_{i=1}^{m} L(\hat{y}_i, y_i)$

Backpropagation / Backward pass: Calculate gradient of the Cost fn. wrt the weights and biases.

$\frac{d}{dx} f(g(x))$

Neural Net Architectures:

Conv Nets (Image)
RNN (Speech)

Deep Q Network

Playing Atari with Deep Reinforcement Learning by Mnih, Kavukcuoglu, and Silver et al.

&

Human-level control through deep reinforcement learning

Agent Uses RL in Real-World Situations

- Agent sees what a human sees
  - No hand crafted features
- Agent has access to high-dimensional sensory inputs
  - Must derive efficient representations of the input
  - Be able to generalize past experience to new situations
- Authors propose a Deep-Q network
  - Test on Atari games
  - Agent takes raw video frames as input

Challenges Using DL with RL

- DL applications use lots of labelled training data
- In RL, we have scalar, often delayed reward
- Mapping of inputs and their labels (values) isn’t straightforward

Challenges Using DL with RL

- DL applications assume training samples to be independent
- In RL, states observed together are likely correlated
- Also, the distribution of the data changes in RL as the agent learns
- The papers address these challenges

Choice of State?

Goal is to Find the Optimal Q-values

- Use a function approximator
- Iteratively find the optimal parameter vector
  - Recall the problem...

Given $\bar{w}_t$:

$$\text{minimize}_{\bar{w}} \; \mathbb{E}\left[\left(R_{t+1} + \gamma \hat{v}(S_{t+1}; \bar{w}_t) - \hat{v}(S_t; \bar{w}) \right)^2 \right]$$

$\hat{v}(S_{t+1}; \bar{w}_t) \rightarrow \hat{q}(S_{t+1}, A_{t+1}; \bar{w}_t)$

$\hat{v}(S_t; \bar{w}) \rightarrow \hat{q}(S_t, A_t; \bar{w})$

Experience Replay

- Store in a ***replay memory*** a set of agent's experiences (s,a,r,s') at any time step
  - Experiences generated using behavior policy

$S_t\ A_t\ R_{t+1}\ S_{t+1}$

- Randomly select a *mini-batch* of experiences from the replay memory to update the weights
  - Breaks temporal correlations

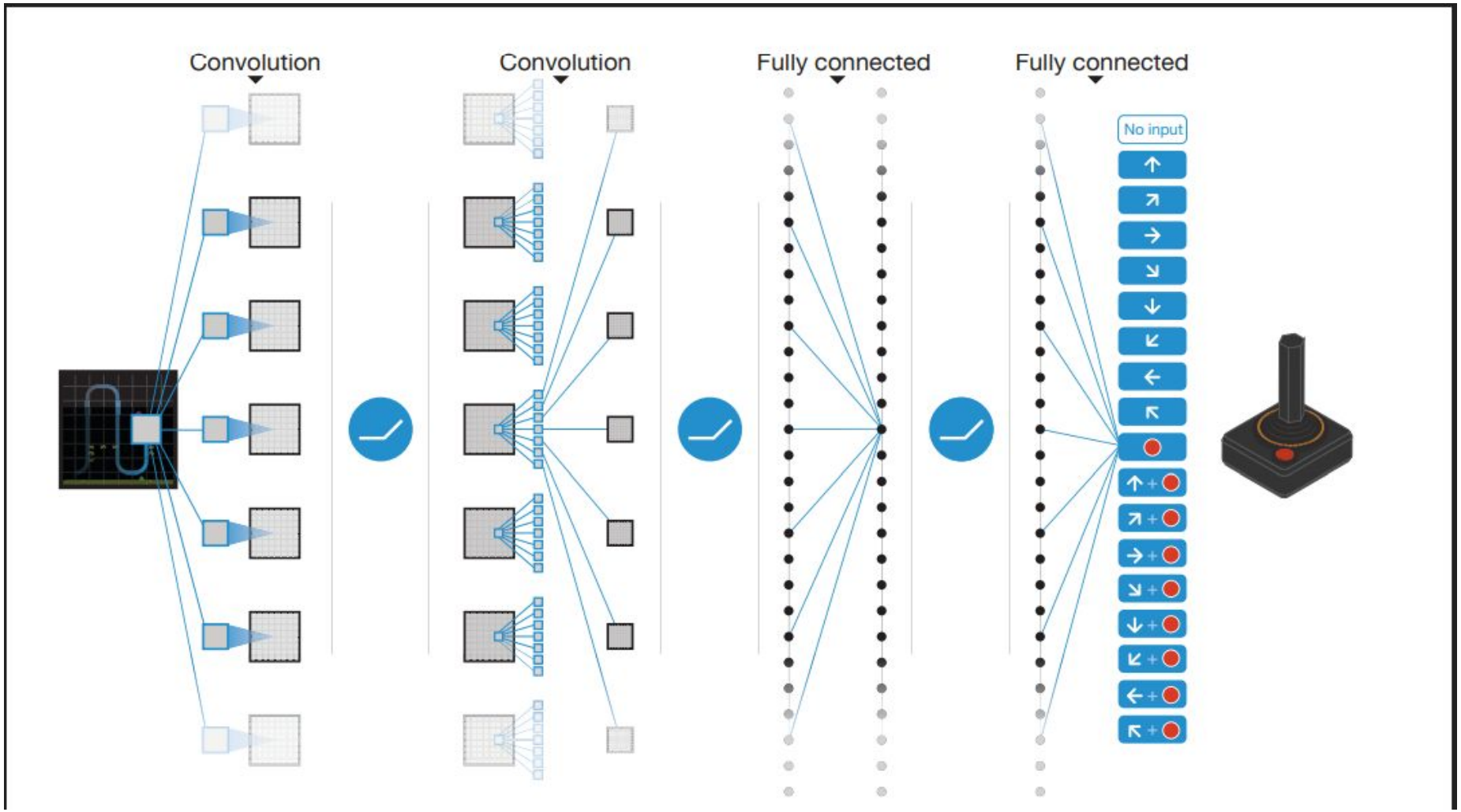
Mini-batch Gradient Updates

Algorithm (Mnih et al.)

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Model Architecture (Mnih et al.)



Policy Gradient Methods

Policy Gradient

- Learn a parametrized policy
- Don't need a value function to pick an action

$$\pi(a|s,\theta) = P[A_t = a|S_t = s,\theta]$$

- Value function may still be used to update the policy parameter
  - The performance measure we would like to maximize

Policy Gradient

- Performance measure $J(\theta)$
- The gradient ascent step is

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)}$$

- As in the gradient bandit algo, we only have noisy estimates of $J(\theta_t)$
- A desirable property is …

Example soft-max policy parametrization

$$\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_a e^{h(s,a,\theta)}}$$

- $h(s,a,\theta)$ is a numerical preference for the state-action pair
  - Could be a neural net
- Soft-max allows the policy to approach a deterministic policy
- We require the policy function to be differentiable wrt θ

Action-Value Function as a Preference Function?

When Compared to epsilon-greedy

- epsilon-greedy restricts the nature of policies

When Compared to epsilon-greedy

- Change in greedy action can cause large changes to the policy PMF
  - Large changes in the policy space

- Soft-max action preference based policies change in a smooth manner

What All Do We Need>

- A differentiable policy parametrization
- A performance measure that we want to maximize
- A way to calculate its gradient
  - Given by the Policy Gradient Theorem
- In practice, we calculate the gradient of a (noisy) estimate of the performance measure

Performance Measure We Want To Maximize

$$\sum_s v_{\pi_\theta}(s)\, p[S_0 = s]$$

Value function

$$J(\theta) \triangleq v_{\pi_\theta}(s)$$

Policy Gradient Theorem tells us how to calculate the gradient

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a \nabla \pi_\theta(a|s) q_{\pi_\theta}(s,a)$$

Proof of the Policy Gradient Theorem

$$J(\theta) = V_{\pi_\theta}(s) = \sum_a q_{\pi_\theta}(s,a)\, \pi_\theta(a|s)$$

$$\nabla_\theta V_{\pi_\theta}(s) = \sum_a \Big(\underbrace{\nabla_\theta q_{\pi_\theta}(s,a)\, \pi_\theta(a|s)}_{f} + \nabla_\theta \pi_\theta(a|s)\, q_{\pi_\theta}(s,a) \Big)$$